ĐẠI HỌC ĐÀ NẴNG
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG VIỆT - HÀN
Vietnam - Korea University of Information and Communication Technology

BÀI GIẢNG
LẬP TRÌNH MẠNG

PGS.TS. Huỳnh Công Pháp; Nguyễn Anh Tuấn;
Lê Thành Công; Nguyễn Thanh Cẩm, Lê Tân
Khoa Khoa học máy tính

http://vku.udn.vn/

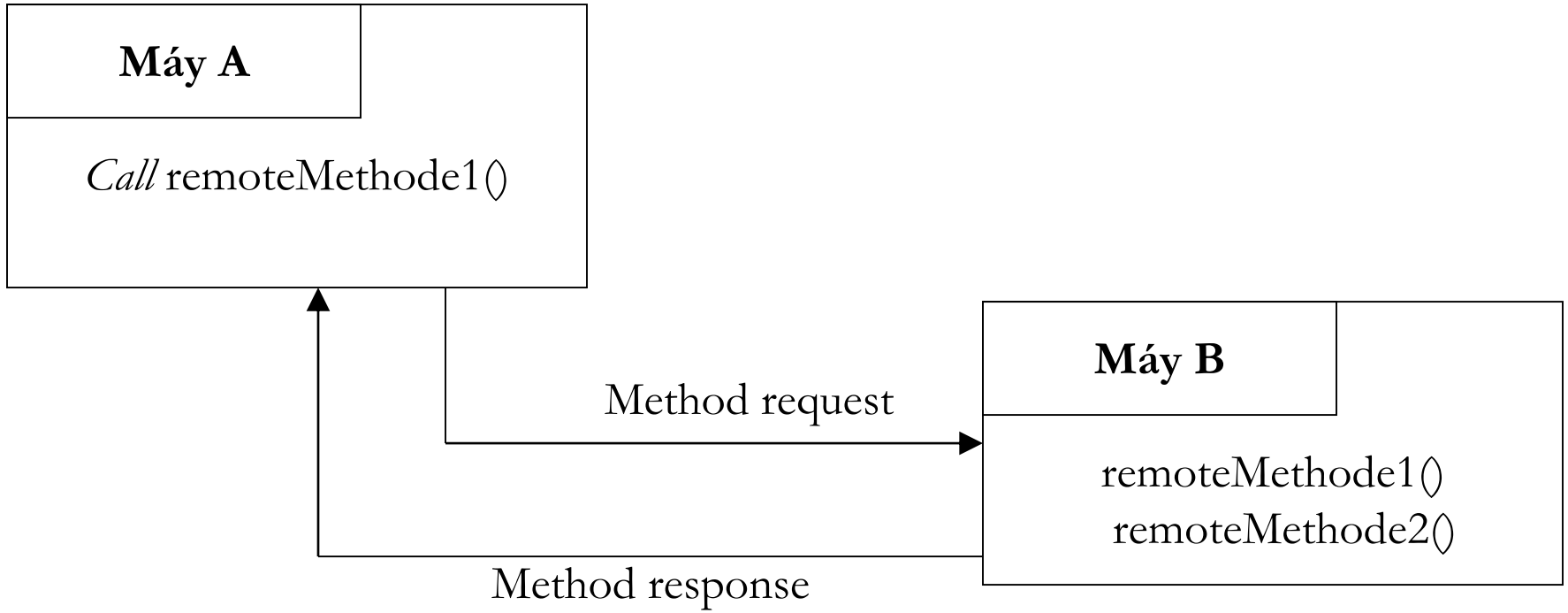
Bài 8. Gọi hàm và thủ tục từ xa

3 kỹ thuật phổ biến:

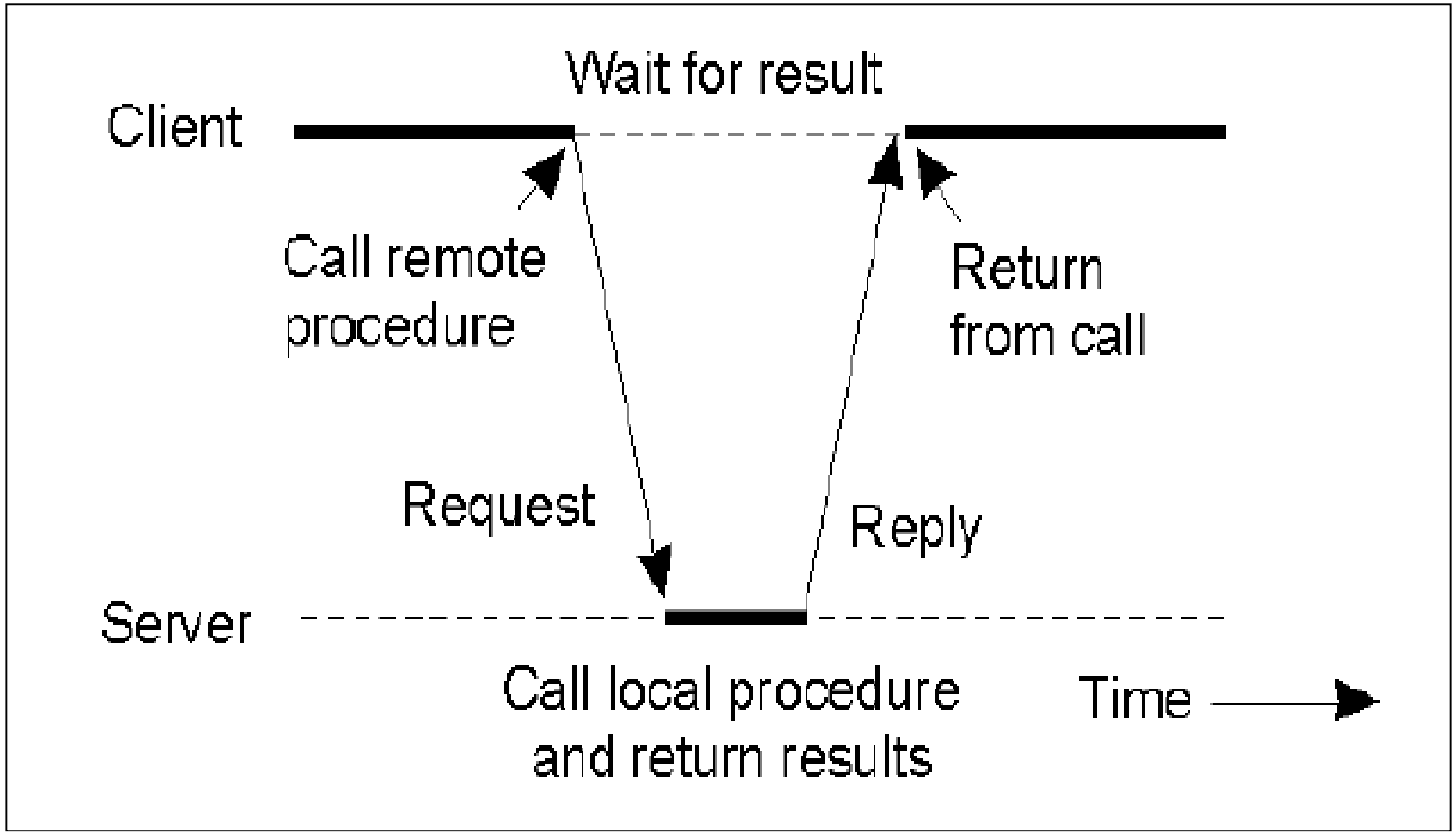
- Remote Procedure Calls (RPC)
- Remote Method Invocation (RMI)
- Common Object Request Broker Architecture (CORBA)

Tổng quan

- Một trong những kỹ thuật độc đáo tạo các chương trình phân tán client/server là cho phép
  - một ứng dụng trên một máy có thể gọi các phương thức ở các ứng dụng trên máy khác trên mạng.
- Công nghệ này hết sức quan trọng cho việc phát triển các hệ thống lớn, bởi vì khả năng phân bố tài nguyên và xử lý bài toán trên nhiều máy khác nhau.



Nguyên lý gọi hàm và phương thức từ xa giữa chương trình client và server.



Sự khác với gọi hàm và thủ tục cục bộ

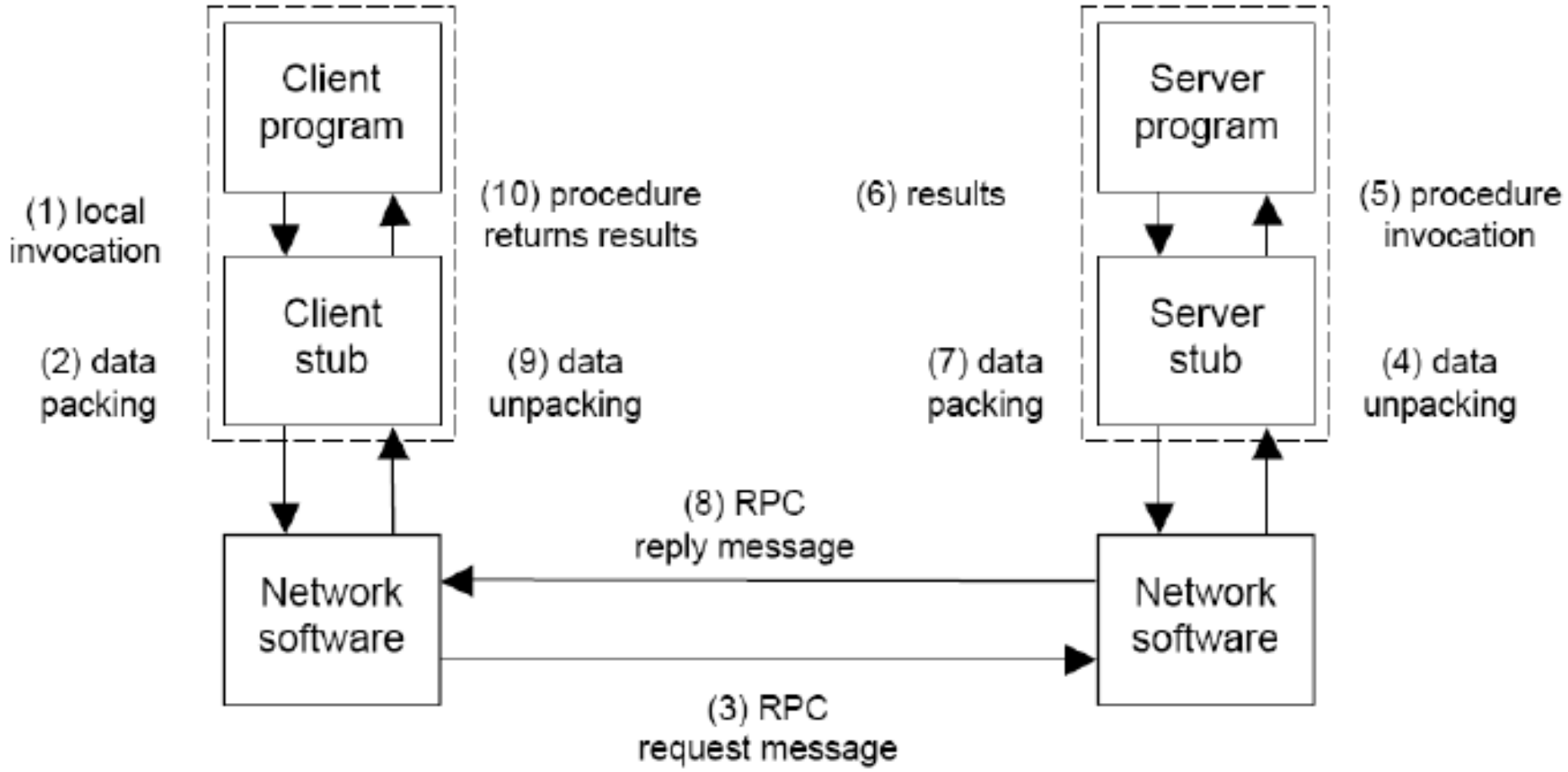
- Độ trễ tương đối lớn hơn
- Không thể truyền con trỏ đến các cấu trúc dữ liệu
  - client và server hoạt động ở các không gian địa chỉ khác nhau
- Môi trường xử lý khác nhau (mở files, mở sockets, etc)
- Chế độ báo lỗi khác nhau

So sánh kỹ thuật RMI với RPC

- RMI chỉ hỗ trợ ứng dụng viết bằng Java.
- RMI làm việc với đối tượng, và cho phép các phương thức chấp nhận và trả lại các đối tượng cũng như kiểu dữ liệu cơ bản.

- RPC hỗ trợ đa ngôn ngữ
- RPC không hỗ trợ khái niệm đối tượng.
- Các thông điệp truyền đến dịch vụ RPC được biểu diễn bởi ngôn ngữ External Data Representation (XDR).
- Chỉ có các kiểu dữ liệu có thể định nghĩa bởi XDR có thể truyền, chủ yếu là kiểu cơ bản, không cho phép đối tượng được truyền.

Các bước gọi thủ tục từ xa RPC

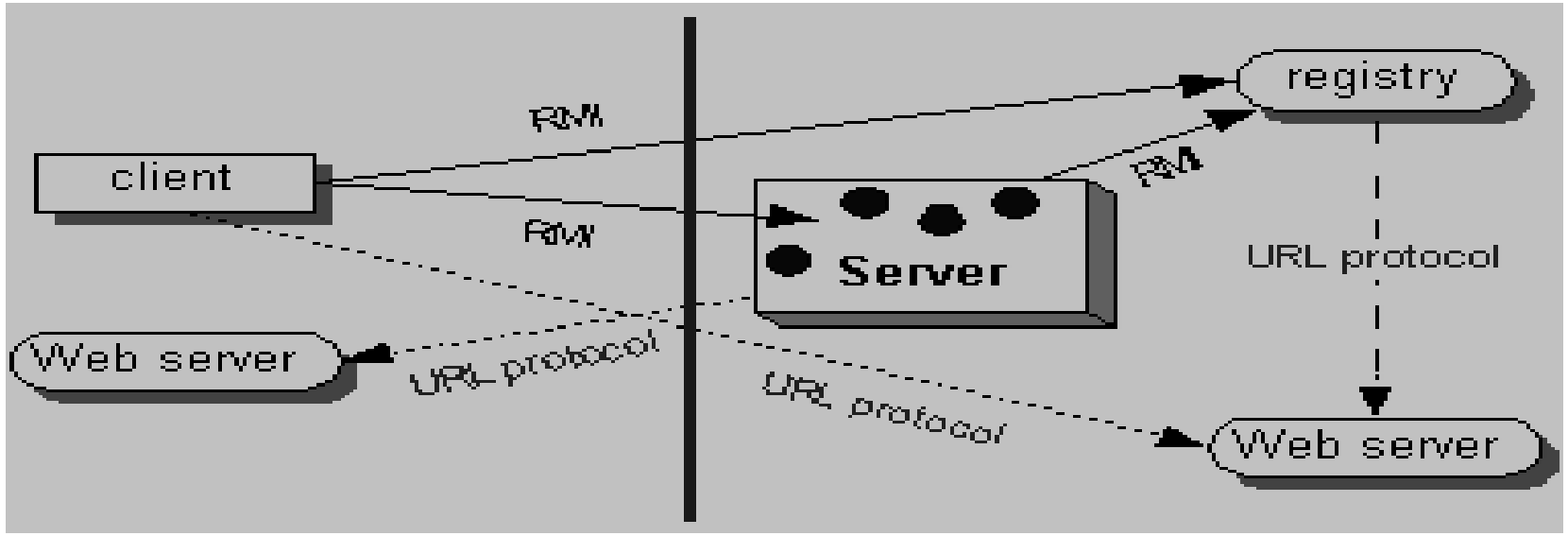


RMI

- RMI cho phép các đối tượng đang thực thi trên một trạm có khả năng gọi các phương thức của các đối tượng khác đang chạy trên các trạm khác.
- Sự gọi các phương thức trên các máy khác được gọi là sự gọi phương thức từ xa (remote method invocation).
- Các phương thức ở xa thực hiện việc tính toán và trả lại kết quả cho các hàm cục bộ.
- RMI là công nghệ Java cho phép tạo các ứng dụng phân tán.

RMI

- Java cung cấp một số interface và lớp cho phép tạo ứng dung phân tán RMI
- Các đối tượng chứa các phương thức có thể được gọi bởi các máy ảo khác được gọi là **đối tượng từ xa (remote objects)**

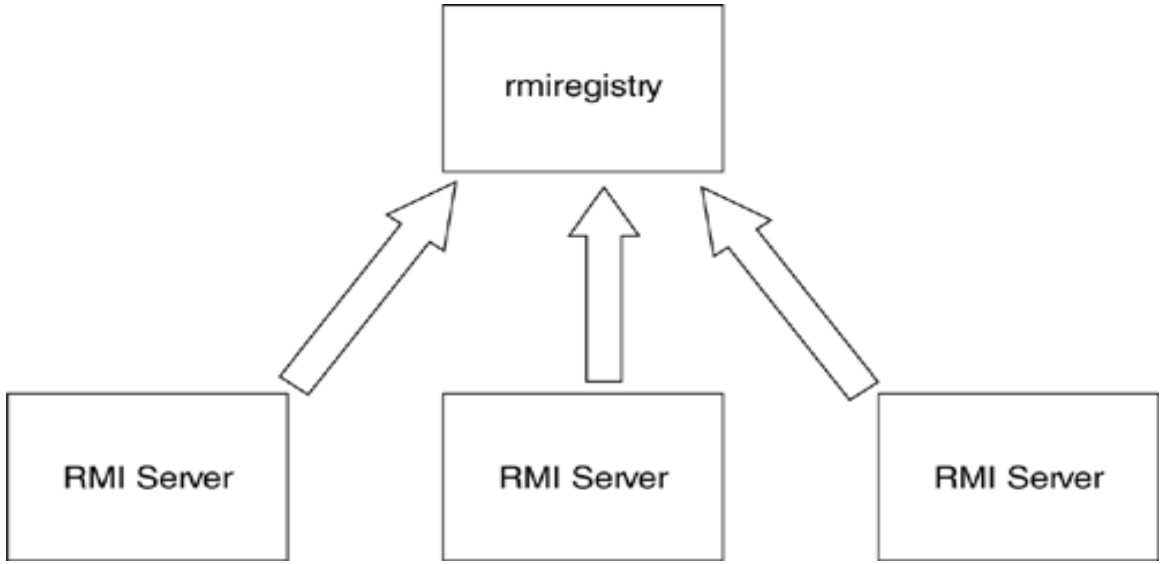


Đối tượng cục bộ và đối tượng từ xa

- Đối tượng cục bộ (Local objects) là các đối tượng được thực thi trên một trạm client.
- Đối tượng từ xa (**remote objects)** là các đối tượng được thực thi ở các máy server ở xa.
- RMI sử dụng các stubs và skeletons
- **stub** là một đối tượng cục bộ hoạt động như proxy cục bộ
- **Skeleton** là sự đại diện cho đối tượng từ xa được đặt trên cùng một trạm.
- Stubs và Skeletons truyền thông với nhau thông qua tầng Remote Reference
- Stubs liên lạc với skeleton sử dụng TCP.
- Đối tượng gọi phương thức ở xa gọi là đối tượng client.
- Đối tượng chứa phương thức được gọi từ xa gọi là đối tượng **Server.**
  - Đối tượng client và server không hoạt động trực tiếp với nhau

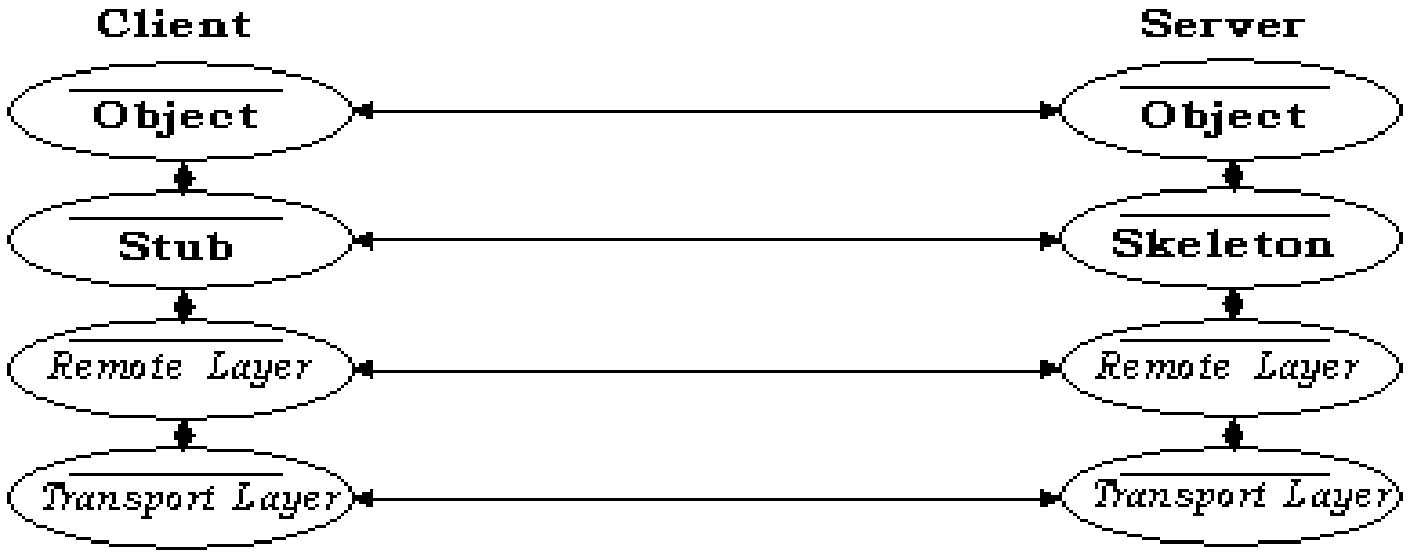
Cách hoạt động của RMI

- Các server RMI phải đăng ký với dịch vụ tra cứu (lookup service, Remote Registry Server), để cho phép các clients tìm thấy chúng.
- Dịch vụ tra cứu này trong Java là *rmiregistry. Dịch vụ* này hoạt động như một tiến trình tách biệt và cho phép các ứng dụng đăng ký các dịch vụ RMI.
- Sau khi một server đã đăng ký, sẽ chờ các yêu cầu RMI từ các client.



Sự phân lớp của RMI

- Đối tượng client gọi các phương thức của stub
- Stub sử dụng tầng remote để liên lạc với skeleton
- Tầng remote sử dụng tầng transport để thiết lập kết nối giữa client và server



Lập trình RMI

- Lập trình RMI liên quan đến 2 bước:
  - Lập trình RMI trên trạm ớ xa
  - Lập trình RMI trên trạm cục bộ

Các bước lập trình RMI trên trạm ở xa

- **Bước 1**

  Tạo một remote interface:
  - Phải là public và phải extend từ interface tên là Remote
  - Khai báo các hàm sẽ được gọi từ xa. Các hàm phải throws RemoteException
- **Bước 2**

  Tạo một lớp hiện thực remote interface:
  - Lớp này cũng phải extends lớp UnicastRemoteObject để kế thừa các chức năng đối tượng từ xa
  - Lớp này tạo và khởi tạo đối tượng remote object
  - Hiện thực tất cả các methods đã định nghĩa trong remote interface
- Tạo hàm **main( )** thực thi lớp ở xa

Các bước lập trình RMI trên trạm ớ xa (tt)

- **Bước 3**

  Tạo các lớp stub và skeleton sử dụng lệnh **rmic**.

- **Bước 4**

  Chép remote interface và file stub sinh ra đến trạm client.

- **Bước 5**

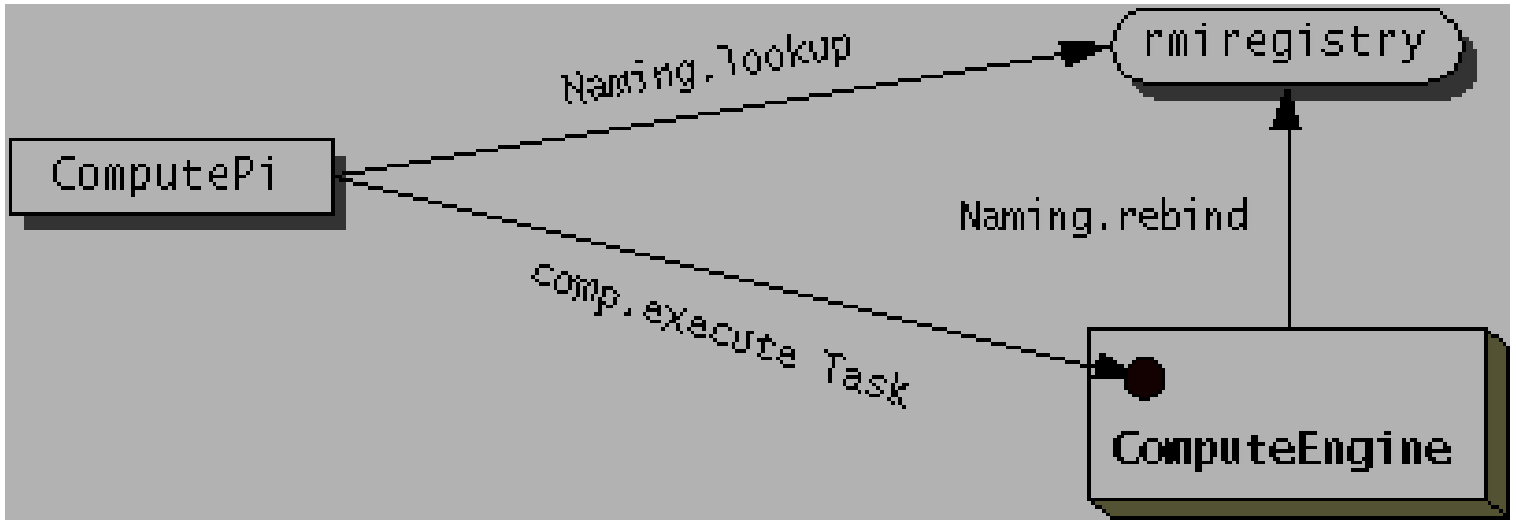
  Khởi động Remote Registry Server sử dụng lệnh **start rmiregistry.**

- **Bước 6**

  Thực thi chương trình dùng lệnh **java** để tạo một đối tượng của lớp. Đối tượng từ xa phải được đăng ký với Remote Registry Server

Lập trình RMI trên trạm cục bộ

- Sau khi RMI được lập trình ở máy từ xa, remote server phải được chạy.
- Tạo một chương trình client để gọi đối tượng từ xa, và hiển thị kết quả



Mã Java: tạo remote interface

- Client nhập câu chào gởi cho server hiển thị
- Server nhập câu trả lời gởi cho client hiển thị

```
import java.rmi.*;
public interface Inter extends Remote {
    String sendMsg(String msg) throws RemoteException;
    //Khai báo thêm các phương thức khác nếu có
}
```

Tạo một lớp hiện thực remote interface

```
import java.rmi.RemoteException;
public class RMIImpl extends UnicastRemoteObject implements Inter {
    public String sendMsg(String msg) throws RemoteException {
        System.out.println(msg);
        String s = “From server: ” + scanner.nextLine();
        return s;
    }
    //Triển khai thêm các phương thức khác nếu có
}
```

Tạo một RMI Server

```
import java.rmi.*; import java.rmi.server.*; import java.rmi.registry.*;
public class Server{
    public static void main(String args[]) {
        try {
            RMIImpl obj = new RMIImpl();
            LocateRegistry.createRegistry(1099); //Nếu không chạy rmiregistry
            UnicastRemoteObject.exportObject(obj, 0); //Nếu không thừa kế UnicastRemoteObject
            Naming.rebind("rmi://localhost:1099/myObj", obj);
        } catch (Exception e) {
            System.err.println("Server exception: " + e.toString());
        }
    }
}
```

Tạo một RMI Client

```
import java.rmi.*;
public class Client {
    public static void main(String[] args) {
        try {
            Inter obj = (Inter) Naming.lookup("rmi://localhost:1099/myObj");
            String s = “From client: ” + scanner.nextLine();
            System.out.println(obj.sendMsg(s));
        } catch (Exception e) {
            System.err.println("Client exception: " + e.toString());
        }
    }
}
```

Tạo các lớp Stub và Skeleton

- Tại cửa sổ dòng lệnh gõ:
  - rmic RMIImpl
  - sẽ có 2 files sinh ra cùng thư mục chứa RMICalcImpl :
    - RMIImpl_Stub.class
    - RMIImpl_Skel.class
  - Chép interface Inter và RMIImpl_Stub.class đến máy client.

Chạy chương trình RMI Server, RMI Client

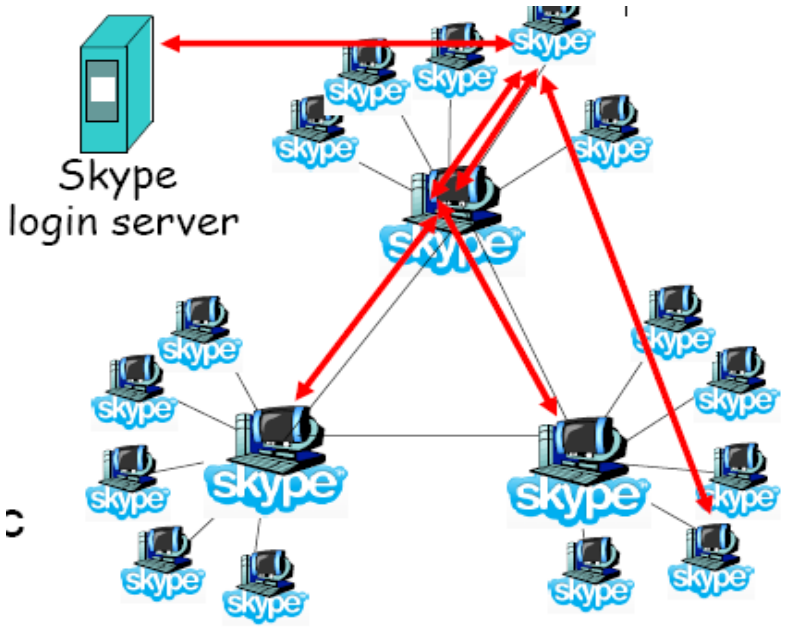
1. Tại cửa sổ dòng lệnh gõ: **start** rmiregistry
2. Dịch Inter, RMIImpl, Server và chạy Server
3. Dịch và chạy Client

Lập trình CORBA

SV tự tìm hiểu

Thảo luận cách hoạt động của Skype

- A chạy ứng dụng chat client trên máy có kết nối internet
  - Ứng dụng hiển thị danh sách friends có chứa B
  - A khởi tạo một kết nối trực tiếp đến B (hoặc ngược lại)
  - A và B bắt đầu chat với nhau trực tiếp



Bài tập gọi hàm và thủ tục từ xa

- Xây dựng chương trình chat bằng RMI
- Chạy chương trình ví dụ trong slides
- Client chat với server: gởi đi 1 câu, nhận về 1 câu (lặp lại nhiều lần)
- Client chat với server: có thể gởi và nhận liên tiếp nhiều câu (callback)

Phụ lục: Truyền tham số trong RMI

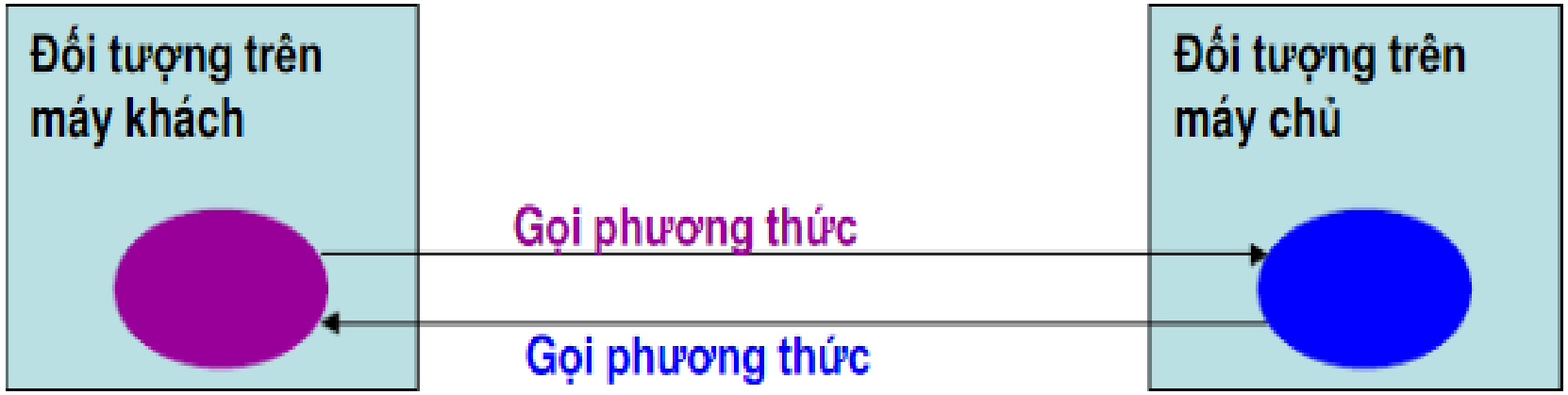
- Đối với việc truyền tham số qua mạng theo cơ chế RMI thì có hai nguyên tắc:
  - Các kiểu dữ liệu nguyên thuỷ như int, char,… đều được truyền theo tham trị.
  - Các dữ liệu kiểu đối tượng buộc phải cài đặt một trong hai giao tiếp là Remote hoặc Serializable.
    - Cài giao tiếp Remote: truyền theo tham chiếu
    - Cài đặt giao tiếp Serializable: truyền theo tham trị.
    - Các đối tượng không cài đặt giao tiếp Serializable hay Remote thì không thể dùng làm tham số truyền qua mạng

Truyền đối tượng theo tham trị

- Giao tiếp Serializable để báo hiệu:
  - Tất cả các đối tượng cài đặt giao tiếp Serializable đều có khả năng tuần tự hóa.
- Khi gọi một phương thức của đối tượng từ xa, nếu trong lời gọi có tham số kiểu đối tượng:
  - Đối tượng sẽ được đóng gói và chuyển toàn bộ đến server (nơi tiếp nhận và thực thi phương thức).
  - Tại server, đối tượng được chuyển về lại trạng thái ban đầu và được sử dụng

Truyền đối tượng theo tham chiếu

- Khi đối tượng cần truyền đi quá lớn -> đóng gói chuyển qua mạng sẽ ảnh hưởng tốc độ thực thi chương trình, server thì không gọi được phương thức trên máy client.
- Để server có thể tham chiếu và xử lý trực tiếp đối tượng đang nằm trên máy client -> chuyển đối tượng theo tham chiếu từ client đến server -> cơ chế callback.



Truyền đối tượng theo tham chiếu

1. Đặc tả giao tiếp cho đối tượng Client
2. Đặc tả giao tiếp cho đối tượng Server
3. Cài đặt chi tiết cho đối tượng Client
4. Cài đặt chi tiết cho đối tượng Server
5. Xây dựng chương trình cho đối tượng Server trên Server và đăng ký với Registry
6. Thiết kế chương trình Client gọi phương thức của đối tượng Server
7. Tạo lớp trung gian (stub và Skeleton) cho cả 2 đối tượng Server và CLient
8. Khởi động rmiregistry
9. Chạy trình chủ trên Server
10. Chạy trình khách trên Client

Ex: Đặc tả giao tiếp cho đối tượng Client, Server

```
import java.rmi.*;
public interface AtClient extends Remote {
    //pt này được gọi bởi đ.tượng AtServer trên máy chủ
    public void callClientMethod(String message) throws RemoteException;
}
```

```
import java.rmi.*;
public interface AtServer extends Remote {
    //Tiếp nhận tham chiếu đến đối tượng AtClient trên trình khách
    public void registerClient(AtClient c) throws RemoteException;
    //Cung cấp dịch vụ cho các trình khách
    public void callServerMethod(String message)throws RemoteException;
}
```

Ex: Cài đặt chi tiết cho đối tượng Client

```
import java.rmi.*;
public class AtClientImpl implements AtClient {
    @Override
    public void callClientMethod(String mes) throws RemoteException{
        System.out.println(mes);
    }
}
```

Ex: Cài đặt chi tiết cho đối tượng Server

```
import java.rmi.*;
public class AtServerImpl implements AtServer {
    AtClient client;//lưu giữ tham chiếu đến đ/tượng trên máy khách
    @Override
    public void registerClient(AtClient c) throws RemoteException {
        client = c;
    }
    @Override
    public void callServerMethod(String mes)throws RemoteException {
        System.out.println(mes);
        for (int i = 1; i < 10; i++){
            String msg= "Server response " + i;
            //triệu gọi phương thức của đối tượng máy khách
            client.callClientMethod(msg);
        }
    }
}
```

Ex: Xây dựng chương trình Server

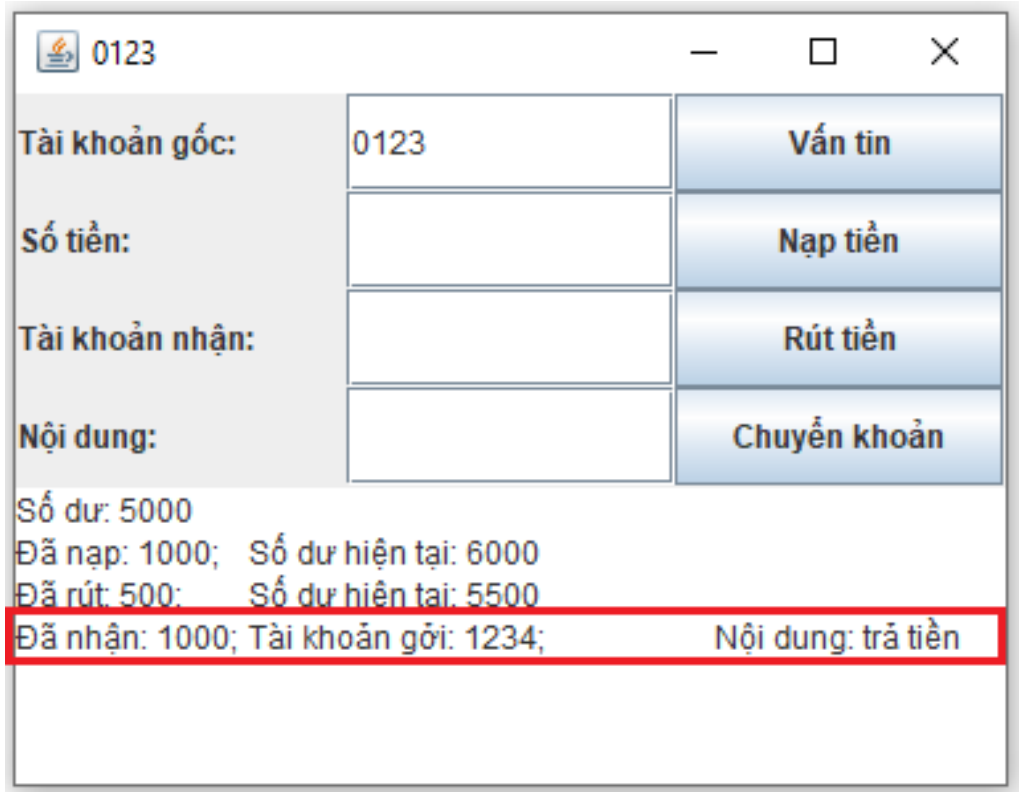
```
import java.rmi.*;
import java.rmi.registry.LocateRegistry;
import java.rmi.server.UnicastRemoteObject;
public class Server{
    public static void main(String args[]) throws Exception{
        //Khởi tạo đối tượng chủ
        AtServer server=new AtServerImpl();
        //thông báo khả năng giao tiếp được từ xa, or extends
        UnicastRemoteObject.exportObject(server, 0);
        //Đăng ký đối tượng với  RMI
        LocateRegistry.createRegistry(1099);
        Naming.bind("rmi://localhost/srvobj", server);
        System.out.println("Waiting for client request ...");
    }
}
```

Ex: Xây dựng chương trình Client

```
import java.rmi.*;
import java.rmi.server.UnicastRemoteObject;
public class Client {
    public static void main(String args[]) throws Exception{
        //Khởi tạo đối tượng khách
        AtClient client = new AtClientImpl();
        //thông báo khả năng giao tiếp được từ xa
        UnicastRemoteObject.exportObject(client, 0);
        AtServer server = (AtServer)Naming.lookup("rmi://localhost/srvobj");
        //Truyền t.chiếu của AtClient từ trình khách lên chủ
        //Trình chủ ko cần .lookup để truy tìm tc đến AtClient
        server.registerClient(client);
        server.callServerMethod("Client Contact Server");
    }
}
```

Bài tập lập trình RMI

- **Lập trình RMI mô phỏng eBanking đơn giản:**
  - Server lưu trữ thông tin clients (số tài khoản, số dư …) trong loại file tuỳ chọn (text, csv, xml, json…)
  - GUI Client có thể vấn tin tài khoản, nạp tiền, rút tiền, chuyển khoản…
  - Server gởi thông báo cho client khi nhận chuyển khoản (callback)



Thank you listening